A Fog-to-Cloud Pipeline for Continuous Offshore Wind Turbine Condition Monitoring

Vishvaksen Machana
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
Student ID: X25173421
x25173421@student.ncirl.ie

***Abstract*—Offshore wind turbines operate in a setting where periodic manual inspection is slow, costly, and weather dependent, yet a developing mechanical fault can escalate between visits. This paper presents a working fog-to-cloud monitoring pipeline that watches two turbines continuously. Five sensor streams per turbine (wind speed, blade vibration, generator temperature, power output, and gearbox pressure) are sampled locally and dispatched to a fog gateway running at the edge of the site. The gateway folds each stream into a fixed time window, computes summary statistics, evaluates condition rules, and forwards a compact aggregate per window to the cloud. In the cloud, a message queue decouples ingestion from an event-driven function that persists each record to a managed key-value store; a second, independently deployed function serves a dashboard programming interface behind a managed gateway, and the operator interface is delivered as static content from object storage. The whole stack is implemented in Node.js and provisioned to a real cloud account in a single infrastructure-as-code apply. A pre-deployment audit found and corrected three defects before any reached the live account. Live measurement against the deployed endpoint shows the pipeline reporting healthy end to end within about a minute of start-up, the stored record count climbing across successive polls, and real per-turbine data rendering in a browser.**

***Index Terms*—fog computing, edge computing, wind turbine condition monitoring, serverless, message queue, NoSQL, Internet of Things.**

I. Introduction

Wind now supplies a large and growing share of grid electricity, and offshore installations are attractive because winds over open water are stronger and steadier than on land. That advantage comes with a maintenance penalty. An offshore turbine is difficult and expensive to reach, access depends on sea state, and a technician visit may be scheduled only at long intervals. Between visits, a bearing can begin to overheat, a gearbox can start to lose lubrication pressure, or a blade can develop a vibration signature that grows steadily worse. If the first evidence of such a fault is a scheduled inspection weeks later, the opportunity to intervene early, while the repair is still small, has already passed.

Condition monitoring addresses this by watching machine health continuously instead of episodically. Surveys of the field describe a broad set of techniques, from vibration analysis and oil debris monitoring to the reuse of the supervisory control and data acquisition signals a turbine already produces [1]. A recurring finding is that continuous, automated monitoring detects incipient faults earlier than periodic inspection and that the sensing itself can be made cost effective when the analysis is designed carefully [2]. The remaining question for a distributed-systems designer is where the analysis should run and how the data should travel from the machine to the people who act on it.

Sending every raw reading straight to a central cloud is wasteful and fragile. A turbine array can emit readings faster than they are individually useful, the link to shore has limited and variable bandwidth, and a monitoring view that stops working whenever connectivity dips is of little value during exactly the storms that stress the machines most. Edge and fog computing offer a better division of labour by pushing computation toward the data source [3]. Fog computing, in particular, positions a compute and networking tier between the sensors and the cloud so that summarisation, filtering, and time-critical rule evaluation happen close to the equipment, while the cloud retains the roles it is best at: durable storage, elastic query serving, and a globally reachable interface [4]. The cloud services used here follow the standard on-demand, elastic, measured model of cloud computing [5].

This paper reports the design, implementation, and live deployment of a fog-to-cloud pipeline for offshore turbine condition monitoring. The concrete deployment, presented under the name North Bank Offshore Array, watches two turbines, each carrying five sensors. The objectives are as follows. First, to move raw-reading processing to a fog tier at the site so that only compact, per-window aggregates cross the link to the cloud. Second, to build a cloud back end that ingests those aggregates reliably, stores them durably, and serves them to an operator dashboard without the ingestion path and the query path interfering with one another. Third, to evaluate the alert conditions that matter for turbine health at the edge, so that a structural, thermal, aerodynamic, or lubrication problem is flagged in the same cycle in which it is observed. Fourth, to deploy the whole system to a real cloud account, not merely a local emulator, and to verify it end to end against the live endpoint. The remainder of the paper describes the architecture and the design decisions behind it, the implementation and its automated tests, the deployment and the defects corrected before it went live, an evaluation using measurements taken

from the running system, and a concluding reflection.

II. System Architecture and Design

The system is a linear pipeline with a branch for the operator view. Sensors produce readings; a fog gateway at the site aggregates and screens them; a message queue carries aggregates to the cloud; an event-driven function stores them; and a separate function reads the stored data back out to a dashboard. Fig. 1 shows the end-to-end topology as deployed. This section walks through each tier and then weighs the main alternatives that were considered and set aside.

A. Sensor and Fog Layer

Each turbine is represented by five independent sensor processes, one per measured quantity: wind speed in metres per second, blade vibration in millimetres, generator temperature in degrees Celsius, power output in kilowatts, and gearbox pressure in bar. A sensor generates values with a bounded random walk: each new reading steps a small random amount away from the previous one and is clamped to a physically plausible range, so the stream drifts realistically rather than jumping arbitrarily. Every sensor is driven by two independently configurable intervals. The first controls how often a new value is produced, and the second controls how often the accumulated values are dispatched to the gateway. Separating these two rates lets a fast-changing quantity be sampled frequently while still being sent to the edge in economical batches, and it mirrors real deployments in which sampling and transmission are tuned independently. Each dispatch a sensor process makes to the gateway is a small JSON object naming the sensor type, the turbine site identifier, and the measurement unit, plus the short list of timestamped values accumulated since the previous dispatch; with the default sampling and dispatch cadence that list holds only a handful of entries, so the whole payload comes to a few hundred bytes, well under a kilobyte.

The fog gateway is a small web service that accepts posted batches of readings and maintains one running accumulator per combination of sensor type and turbine. Rather than retain the raw readings, the gateway folds each incoming value immediately into a compact record holding the count, running sum, minimum, maximum, and most recent value, so its memory footprint stays flat regardless of how many readings arrive. On a fixed cadence, the gateway closes the current window: it seals each accumulator into a summary carrying the window bounds, the count, the minimum, the maximum, the mean, and the latest value, then evaluates that summary against the condition rules for its sensor type.

Four alert rules are evaluated, one per safety-relevant sensor type; power output is retained for display and trend analysis but carries no rule, since a high or low instantaneous output is not by itself a fault. Table I lists the rules. Three of them watch the window mean, because a sustained average is a more meaningful signal of structural, thermal, or aerodynamic stress than a single transient spike. The lubrication rule instead watches the window minimum, because a momentary pressure drop is exactly the event that matters for a gearbox: the worst reading in the window, not its average, is the safety signal. Encoding rule evaluation at the edge means that a fault is detected in the same cycle in which the readings that reveal it were observed, without a round trip to the cloud.

TABLE I
Edge-Evaluated Condition Rules

Sensor	Condition (per window)	Alert raised
Blade vibration	mean above 8 mm	Structural risk
Generator temperature	mean above 95 °C	Generator overheat
Wind speed	mean above 25 m/s	High-wind shutdown risk
Gearbox pressure	minimum below 2.5 bar	Lubrication fault
Power output	(informational, no rule)	None

When a window closes, the gateway does not send one message per turbine per sensor. Instead it collects every summary produced in that cycle and hands the whole set to a single batched send toward the queue, splitting the set into chunks only where the queue interface caps the number of entries in one batched call. For a two-turbine array producing up to ten summaries per cycle, this reduces the number of network calls to the cloud from ten to one, which lowers request cost and the per-message overhead of the ingestion path.

B. Queue, Ingestion Function, and Store

Aggregates leave the fog tier through a managed message queue rather than a direct call into the storage function. The queue is a buffer and a shock absorber: it holds messages durably until they are consumed, so a slow or briefly unavailable consumer cannot lose data or push back on the fog gateway, and a burst of aggregates is smoothed into a rate the consumer can sustain. This is the classic decoupling that message-oriented middleware provides between producers and consumers [6]. The decoupling covers the consumer side of the queue, not the hop onto it: if the gateway's own batched send to the queue fails, the window has already been sealed and its accumulators cleared, so there is nothing left to retry against, and the gateway simply logs the failure and moves on to the next window rather than retrying the call or buffering that window's aggregates locally. This is a stated trade-off rather than an oversight, accepted at the data volumes a two-turbine array produces, where the next window arrives a few seconds later.

An event-driven cloud function consumes the queue. It is invoked with a batch of messages, transforms each aggregate into a stored item, and writes it to a managed key-value store. The store uses a two-part key. The partition key is the sensor type, which groups all readings for a given quantity together and makes the common query, "give me the recent windows for this sensor," a single efficient lookup. The sort key concatenates the window-end timestamp with the turbine identifier. The timestamp gives a natural chronological

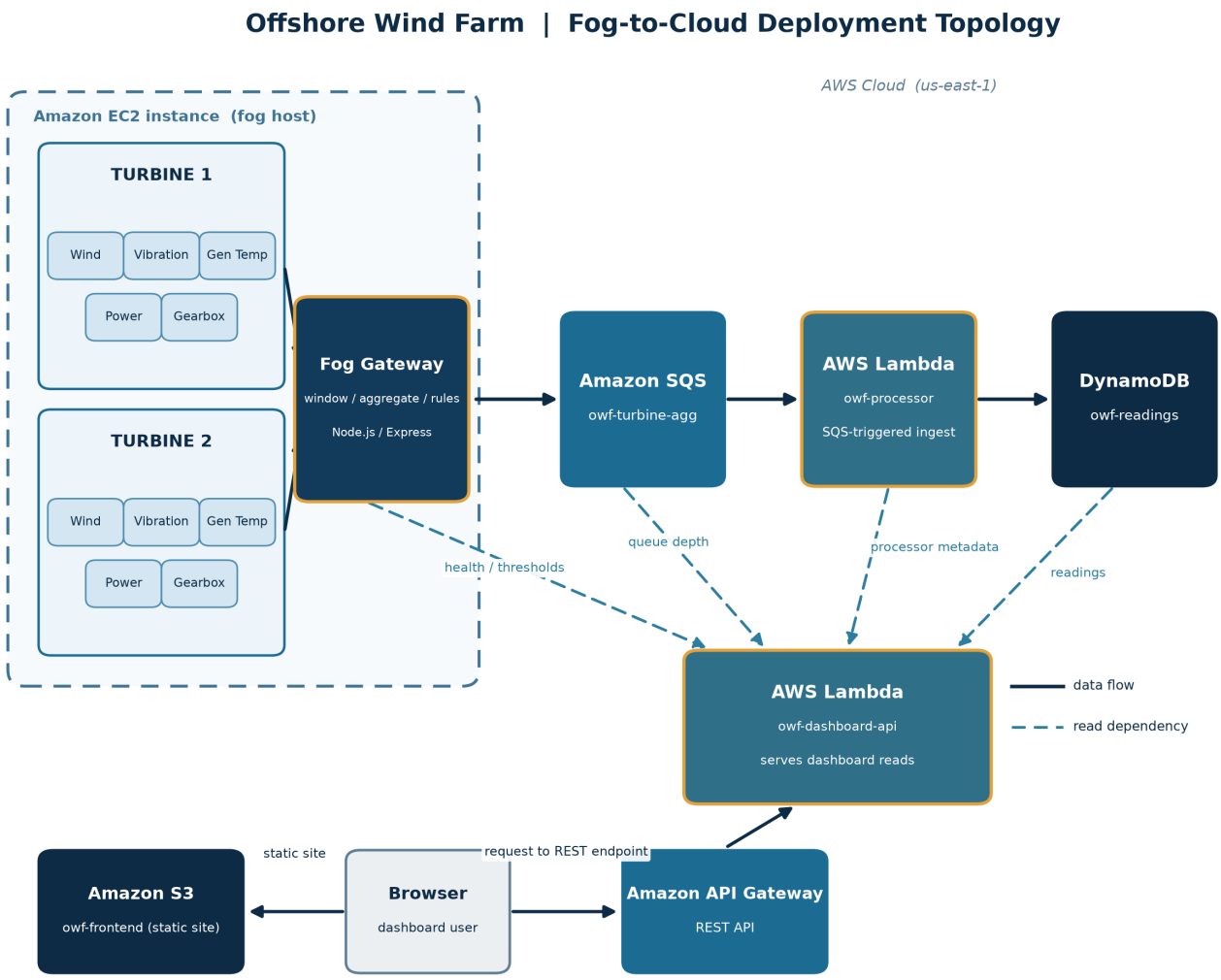

Fig. 1. Fog-to-cloud deployment topology. The fog gateway and the ten sensor processes run in containers on a single edge host. Solid arrows show the ingestion data flow from sensors through the queue and ingestion function into the store. Dashed arrows show the read paths the dashboard function assembles from the gateway, the queue, the ingestion function, and the store. The browser loads static assets from object storage and calls the dashboard programming interface through the managed gateway.

ordering within each partition, and appending the turbine identifier is deliberate: when both turbines close a window for the same sensor in the same cycle, their two items share a partition key and a window-end time, and without the turbine identifier in the sort key the second write would overwrite the first. Concatenating the identifier makes the two keys distinct and lets both turbines coexist in the same partition without collision.

C. Dashboard Programming Interface and Frontend

The operator view is served by a second cloud function that is deployed independently of the ingestion function, so the read path and the write path scale and fail separately. This function answers a small read-only programming interface: a farm-grid endpoint that returns the latest reading for every sensor on both turbines, a readings endpoint that returns a recent series for one sensor and turbine (used to draw the cross-turbine power trend), a health endpoint, and a back-end statistics endpoint. The health endpoint reports four independent booleans (the fog gateway is reachable, the queue is reachable, the ingestion function is active, and the pipeline is flowing, meaning the freshest stored window is recent) together with the age of the freshest window. Assembling this view requires the dashboard function to read from four sources: the store for readings, the queue for depth, the ingestion function for its state, and the fog gateway for its health and its published thresholds. These read paths are the dashed edges in Fig. 1.

The dashboard function is reached through a managed programming-interface gateway that exposes a public REST endpoint. The static frontend (a single page, a stylesheet, the page script, and a locally vendored charting library) is served from object storage. The page renders each turbine as a tile showing all five live metrics and a status beacon, with a shared chart comparing power output across the two turbines, an alert strip naming any flagged condition, and a footer giving the

fleet summary and the four pipeline health indicators. The page script re-fetches the farm grid, the health endpoint, and the back-end statistics together every two and a half seconds and repaints the tiles, the alert strip, and the footer from each response, so the view stays current without a manual reload. The frontend learns which programming-interface origin to call from a single inline value in the page that is substituted with the real gateway origin at upload time and read once by the page script; in local development, where one process serves both the interface and the page, the value is left unset and calls default to the same origin.

D. Critical Analysis of Alternatives

Several designs were weighed and set aside. Each choice below was made for a stated reason, not by default.

Direct function call versus a queue. The fog gateway could invoke the storage function directly and synchronously, removing the queue and one hop of latency. This was rejected. A synchronous call couples the edge to the availability and speed of the cloud function: if the function is throttled, cold, or briefly unavailable, the gateway blocks or drops data, and back pressure propagates all the way to the sensors. The queue breaks that coupling. It absorbs bursts, retries delivery on the consumer's behalf, and lets the fog tier consider an aggregate handed off the instant it is enqueued [6]. The small extra latency the queue adds is irrelevant for condition monitoring on a window cadence, whereas the durability and decoupling it provides are exactly what a lossy edge link needs.

Managed key-value store versus a relational database. A relational database would offer joins, ad hoc queries, and a familiar schema. It was set aside for this workload. The access pattern is narrow and known in advance: append time-ordered aggregates, then read back the most recent windows for a sensor. A key-value store designed for exactly this pattern of high-volume writes and key-based reads delivers predictable performance at scale without the operational weight of a relational engine, which is the trade-off the original Dynamo design articulated and which the managed service inherits [7], [8]. The chosen key scheme turns the one query the dashboard needs into a single partition lookup, so the expressiveness a relational store would add would go unused while its fixed cost would be paid continuously.

Event-driven functions versus an always-on service. The ingestion and interface tiers run as event-driven functions rather than a continuously running server. This suits a workload that is bursty and, for the interface, intermittent: the account pays per request rather than for idle capacity, and the platform handles scaling. The known cost is cold-start latency when a function has been idle and must be initialised before serving a request [9]. For this system the effect is minor: ingestion is driven by a steady stream of queue messages that keeps the function warm, and the dashboard polls on a short interval that does the same, so the paths that matter for the live view rarely pay a cold start.

E. Security Posture

The edge host exposes a single inbound port carrying only the gateway's health and threshold endpoints; it has no remote-login key pair and is administered entirely through the cloud provider's systems-management channel, which removes the most common remote-access attack surface. Both cloud functions run under the account's shared laboratory role because the account cannot create new roles; the design intent, stated plainly for a production setting, is to split this into least-privilege roles, granting the ingestion function only the ability to receive and delete queue messages and write items, and the dashboard function only read-oriented permissions. The dashboard programming interface is intentionally left unauthenticated because it exposes only aggregated, non-personal turbine telemetry; because it is served from one origin and called from another, every response, including error responses, carries an explicit cross-origin header so that a browser will accept the reply under the same-origin rules that govern cross-origin requests [10].

III. Implementation

The entire stack is implemented in Node.js with no build step and no TypeScript. The fog gateway and the local dashboard are built on the Express web framework; the cloud clients for the queue, the store, and the functions come from the official cloud provider software development kit for JavaScript. The dashboard's power-output trend chart is drawn with a charting library that is vendored into the project and served locally rather than from a content delivery network, so the page has no third-party runtime dependency. For local development and continuous integration the full stack runs under Docker Compose against a local emulator of the cloud services, which lets the same code run unchanged against the emulator or against the real account by changing only endpoint and credential configuration.

A. Automated Test Suite

Testing uses the test runner built into the Node.js runtime, with no external test framework. Cloud-facing code is written to accept an injected client, so the unit tests exercise it against small hand-written fake clients rather than against a live cloud service or even the local emulator; this keeps the unit suite fast, deterministic, and free of network dependence. The suite totals 71 tests, distributed across the four modules as shown in Table II. The dashboard total includes twelve tests dedicated to the programming-interface dispatch path described below, which is the most intricate part of the codebase and therefore the part most in need of direct coverage.

B. Continuous Integration

A continuous-integration workflow runs on every push and every pull request as two dependent jobs. The first job installs and runs the unit suites for all four Node.js modules. Only if that job passes does the second job run: it brings up the emulator-backed Docker Compose stack, runs an end-to-end pipeline check that pushes data through the sensors, the

TABLE II
AUTOMATED TEST COVERAGE BY MODULE

Module	Tests
Sensors	8
Fog gateway	25
Ingestion function	7
Dashboard (incl. 12 dispatch-path tests)	31
Total	**71**

gateway, the queue, the ingestion function, and the store, and then tears the stack down. Gating the integration job on the unit job keeps feedback fast for the common case (a logic error is caught by the quick unit suite) while still exercising the assembled system before any change is considered good.

C. Dashboard Function and the Programming-Interface Bridge

Deploying the dashboard interface as a cloud function raised a design choice: either maintain a second, parallel set of routes for the function, or reuse the same interface the local Express application already serves. The implementation reuses the existing application directly. Each event delivered by the managed gateway is bridged into a request and response pair shaped the way the Express application expects, the application handles it exactly as it would a local request, and the resulting response is translated back into the shape the gateway requires. This avoids duplicating business logic across two code paths that could drift apart, and it is why the dispatch path carries its own dedicated block of tests. Every response produced through the bridge, including the error responses, carries the cross-origin header so that the browser accepts replies served from a different origin than the page itself.

D. Real Cloud Deployment and Defects Corrected

The system was deployed live on 16 July 2026 to a real cloud-provider laboratory account (account 015611713565) in the North Virginia region. Provisioning is expressed as infrastructure as code: a single apply, run inside an isolated workspace so that it could not disturb any unrelated state, created 24 resources with no manual command-line steps. The live resources are a key-value table named owf-readings, a message queue named owf-turbine-agg, the ingestion function owf-processor and the dashboard function owf-dashboard-api reached through a REST programming-interface gateway (identifier zwwf3aohya), an edge compute instance (identifier i-0a808bebdd67990f5) whose firewall admits only the single gateway port and which carries no login key pair, a fixed public address (54.227.202.229) attached to that instance, and two object-storage buckets, one serving the dashboard frontend (owf-frontend-015611713565) and one used for deployment staging (owf-deploy-015611713565).

A code audit carried out before deployment found and corrected three defects, none of which reached the live account. Stating them honestly is part of the engineering record.

Defect one: an item-count undercount. The dashboard's routine for counting stored records issued a single count-only scan of the table. A scan of this kind returns only the first page of results, capped by a response-size limit, so once the table grew beyond that limit the reported count would silently understate the true number of records. The fix follows the store's pagination cursor in a loop, issuing a fresh count-only scan for each page and summing the per-page counts until the cursor is exhausted, so the total is correct regardless of table size.

Defect two: a missing batching path. The fog gateway originally sent one message per closed window, one call to the queue per summary. On a real account this multiplies request count and cost without benefit. The fix collects a whole cycle's summaries and sends them in batched calls, splitting only where the queue interface caps the number of entries per batch, so a full cycle for the two-turbine array is delivered in a single call.

Defect three: a credential-gating error. Two cloud-facing modules built static, emulator-only credentials whenever the standard access-key environment variable was present. This is correct against the local emulator, but it is wrong in production, because the serverless platform always injects that same variable to carry the function's own execution-role credentials. As written, both live functions would have discarded their real, complete execution-role credentials in favour of an incomplete static pair and would have failed authentication on their first call to the store or the queue. The fix gates the static-credential branch on the presence of the local-emulator endpoint variable instead, which is the signal that genuinely distinguishes local development from a real deployment. With that change, a live function falls through to the software development kit's default credential chain and resolves the real execution-role and instance-profile credentials the platform provides.

The implementation source is available at [GitHub repository URL].

The system was evaluated by measuring the running deployment, not by inspecting the code alone. All figures below were read from the live endpoint.

E. End-to-End Liveness

Within about a minute of the edge instance finishing its container start-up, the health endpoint reported all four indicators true (the fog gateway reachable, the queue reachable, the ingestion function active, and the pipeline flowing), and the freshest stored window was roughly three seconds old. A freshest-window age in single-digit seconds means that a reading produced at a turbine had already traversed the sensor, the fog gateway, the queue, the ingestion function, and the store, and was being read back by the dashboard, all within seconds. This is the property that distinguishes a live pipeline from a set of individually deployed but disconnected components.

F. Data Accumulation and Real Content

The stored record count was polled repeatedly across the verification window and was observed to climb from 17 to 132

to 204 items, confirming that the ingestion function was consuming the queue and writing to the store continuously rather than once. Over the same window the farm-grid endpoint returned real per-turbine metric data for both turbines: five current metrics each, with their window minimum, maximum, and mean. All four static frontend assets were fetched successfully from object storage, the wildcard cross-origin header was present on the programming-interface responses, and the inline programming-interface origin in the page had been correctly substituted with the real gateway address at upload time, so the browser could reach the interface across origins.

Fig. 2 is a screenshot of the deployed dashboard rendered in a real browser. It shows both turbines' live metric tiles, the cross-turbine power-output trend chart, the fleet summary (two turbines, no active alerts, 204 records archived), and the pipeline footer with all four health indicators reporting up and the freshest window under ten seconds old. The screenshot is evidence that the system is not merely reachable by a command-line probe but renders correct, live content to a human operator.

G. Cost

The design keeps continuous cost low. Every cloud service in the pipeline except the edge instance is billed per request and, at the volumes this deployment produces, stays within the provider's free tier: the queue, the two functions, the key-value store, the programming-interface gateway, and the object storage all bill on use. The edge instance is the only continuously billed resource, and because the serverless dashboard and interface do not depend on it being up (only fresh sensor data and the gateway's own health reading do), it can be stopped between demonstrations without taking the dashboard or the interface offline. This means the standing cost of the system when idle is essentially the cost of one small instance, and even that can be paused.

H. Limitations

Three limitations are stated plainly. First, the sensor and fog tier runs on a single edge instance, which is a single point of failure for that tier: if the instance stops, new readings stop, although the previously stored data and the serverless read path remain available. A production deployment would replicate the fog tier across instances or availability zones. Second, the laboratory account is session based, so only single-session behaviour was verified; multi-day stability, credential rotation over long runs, and behaviour across account-session boundaries were not exercised. Third, both cloud functions currently share one broadly scoped account role, as noted in the security discussion; splitting them into least-privilege roles is understood and specified but was not possible in an account that cannot create new roles.

IV. Conclusion

This work set out to move offshore turbine condition monitoring from periodic inspection toward continuous, automated observation, and to do so with a distributed design that places computation where it belongs: summarisation and rule evaluation at a fog tier close to the machines, durable storage and elastic serving in the cloud. The result is a working pipeline that watches two turbines and five sensors each, flags structural, thermal, aerodynamic, and lubrication faults at the edge in the cycle they appear, and presents a live operator view drawn from a real cloud deployment.

Several lessons stand out. The value of the message queue was not latency but resilience: decoupling the lossy edge link from the cloud consumer is what makes the pipeline tolerant of the conditions offshore monitoring must survive. The sort-key design was a reminder that a small modelling decision, appending the turbine identifier to a timestamp, is the difference between two turbines coexisting and one silently overwriting the other. Most instructive were the three defects the pre-deployment audit caught: the pagination undercount and the missing batching path were correctness and cost issues that a local emulator would never surface, and the credential-gating error would have broken both live functions on their first real call while passing every local test, because the very environment variable it keyed on means opposite things locally and in production. Testing against an emulator is necessary but not sufficient; the audit and the live verification against the real account were what turned a stack that worked locally into one that works deployed. Natural next steps are replicating the fog tier for high availability, splitting the shared role into least-privilege roles, and extending the edge rules from fixed thresholds toward trend-based detection so that a fault is flagged from its trajectory rather than only when it crosses a line.

References

[1] P. Tchakoua, R. Wamkeue, M. Ouhrouche, F. Slaoui-Hasnaoui, T. A. Tameghe, and G. Ekemb, "Wind turbine condition monitoring: State-of-the-art review, new trends, and future challenges," *Energies*, vol. 7, no. 4, pp. 2595–2630, 2014.

[2] W. Yang, P. J. Tavner, C. J. Crabtree, and M. Wilkinson, "Cost-effective condition monitoring for wind turbines," *IEEE Transactions on Industrial Electronics*, vol. 57, no. 1, pp. 263–271, 2010.

[3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. 1st Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12)*, 2012, pp. 13–16.

[5] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-145, 2011.

[6] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.

[7] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, 2007, pp. 205–220.

[8] M. Elhemali, N. Gallagher, N. Gordon, J. Idziorek, R. Krog, C. Lazier, E. Mo, A. Mritunjai, S. Perianayagam, T. Rath, S. Sivasubramanian, J. C. Sorenson, S. Sosothikul, D. Terry, and A. Vig, "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '22)*, 2022, pp. 1037–1048.

[9] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '18)*, 2018, pp. 133–146.

Fig. 2. The deployed operator dashboard rendered live in a browser. Each turbine tile shows all five metrics with their window range and mean; the lower panel compares power output across the two turbines; the footer reports the four pipeline health indicators and the age of the freshest window.

[10] A. Barth, “The web origin concept,” Internet Engineering Task Force, Tech. Rep. RFC 6454, 2011.